

Summer Internship Report
on
“Handwritten Character Recognition using Linear Regression”

Submitted in partial fulfillment of the Requirement for the award of the degree of

Bachelor of Engineering
in
Computer Science and Engineering

by

GUDIPALLY AISHWARYA (1608-22-733-091)

Under the guidance of

Mrs.T. Soumya

Assistant Professor

Department of CSE



Department of Computer Science and Engineering

MATRUSRI ENGINEERING COLLEGE

DECEMBER 2025

MATRUSRI ENGINEERING COLLEGE

SAIDABAD - 500059



Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the internship report entitled “**Handwritten Character Recognition using Linear Regression**” submitted by **Mrs. Gudipally Aishwarya** bearing **H.T. No: 1608-22-733-091**, in the partial fulfillment of the requirement for the award of the degree of **Bachelor of Engineering in Computer Science and Engineering** is a bonafide work carried by him/her.

The results of the investigations enclosed in this report have been verified and found satisfactory.

Internal Supervisor
Dept. of CSE
MECS.

HOD
Dept. of CSE
MECS.

CERTIFICATE

टेलीफोन/PHONE : +91-40-2458 4785
फैक्स/FAX : +91-40-2418 5033
ईमेल/e-mail : bhogendra.drdl@gov.in
ड्रोनामेल/DRONA mail: bhogendra@drdl.hyddom
डॉ. पी. वी. आर. आर. भोगेन्द्रा राव, वैज्ञानिक – जी
Dr. P V R R Bhogendra Rao, Scientist – G
नवीनतम प्रौद्योगिकियाँ और स्वदेशीकरण निदेशकालय
Directorate of Advanced Technologies and Indigenization



भारत सरकार / Government of India
रक्षा मंत्रालय / Ministry of Defence
रक्षा अनुसंधान तथा विकास संगठन /
Defence Research & Development Organisation
रक्षा अनुसंधान एवं विकास प्रयोगशाला /
Defence Research & Development Laboratory
पी.ओ. कंचनबाग / P.O. Kanchanbagh
हैदराबाद, भारत / Hyderabad, India – 500 058

सं./No. DRDL/501/DATI/STU/2025/15

दिनांक/Date: August 1, 2025

CERTIFICATE

This is to certify that the thesis entitled “Handwritten Character Recognition using Linear Regression” that is being submitted by Ms. Gudipally Aishwarya (Roll No. 1608-22-733-091) to the Department of Computer Science and Engineering, Matrusri Engineering College, is record of bona fide work carried out by him in this organization under my guidance and supervision during the period 1st June, 2025 to 31st July 2025.

This is also to certify that the results embodied in this thesis / dissertation have not been submitted to any other university or institute for the award of any degree or diploma.

Dr. PVRB Bhogendra Rao
Scientist – G
Directorate of Advanced Technologies
and Indigenization

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to **Dr. PVRR Bhogendra Rao** Scientist– G Directorate of Advanced Technologies and Indigenization for providing me with the opportunity to undertake a Summer Internship within the organization.

I extend my appreciation to all the individuals who collaborated with me during this internship. Their patience and openness contributed to creating an enjoyable working environment.

With immense pleasure and gratitude, I acknowledge the support and facilities provided by Principal **Dr. G. Amarendar Rao** that enabled me to successfully accomplish this internship.

I am grateful to my Head of the Department, **Dr.T. Raghunadha Reddy** for his constructive criticism throughout my internship, which significantly contributed to my growth.

Special thanks to the Department Internship Coordinator, for their support and advice in securing and completing the internship at DRDL.

I extend my heartfelt thanks to the Department staff members and friends who played a crucial role in assisting me with the successful completion of this internship.

Name of the student
Gudipally Aishwarya

ABSTRACT

Handwritten Character Recognition is a method that helps computers read and understand characters written by humans. In this project, we use **Linear Regression**, a basic machine learning technique, to identify handwritten characters. First, the handwritten images are cleaned and converted into numerical values so the computer can process them. Then, Linear Regression is used to learn patterns from these images and predict which character they represent. Although Linear Regression is mainly used for predicting numbers, we use it here to classify characters by giving each character a unique number. The system works well for simple data and gives a basic idea of how machine learning can be used for handwriting recognition. However, its accuracy is limited compared to more advanced algorithms. This project shows how Linear Regression can be applied in a simple way and helps us understand the basics of character recognition.

Keywords:

Handwritten Character Recognition, Linear Regression, Machine Learning, Feature Extraction, Image Processing, Classification, Pattern Recognition, Dataset, Prediction Model.

TABLE OF CONTENTS

Section No.	Title	Page No.
	CERTIFICATE	ii
	INTERNSHIP COMPLETION CERTIFICATE	iii
	ACKNOWLEDGEMENT	vi
	ABSTRACT	vi
1.	INTRODUCTION	1
2.	BACKGROUND KNOWLEDGE	2
2.1	Linear Regression	3
3.	INTERNSHIP OBJECTIVES AND GOALS	9
4.	TECHNOLOGY STACK	10
5.	ACHIEVEMENTS & CONTRIBUTIONS	12
6.	METHODOLOGY OF PROJECT	13
7.	RESULT SCREENS	
7.1	Environmental Setup	16
7.2	Source Code	19
7.3	Output Screens	21
8.	FUTURE APPLICATIONS	22
9.	CONCLUSION	23

LIST OF FIGURES

FIGURE NO	NAME OF THE FIGURE	PAGE NO
2.1.1	Handwritten Characters	8
5.1	Pixel Example	12
7.1.1.	Anaconda Download	16
7.1.2.	Anaconda Navigator	17
7.1.3.	Jupyter Notebook	18
7.3.1.	Screen 1	21
7.3.2.	Screen 2	21

1. INTRODUCTION

Handwritten Character Recognition is a technology that allows computers to read and understand handwritten digits or letters. It is used in many real-life applications such as digitizing handwritten documents, processing bank cheques, and recognizing numbers in forms. Since everyone writes differently, teaching a computer to identify handwritten characters is a challenging but important task.

In this project, the **MNIST dataset** is used, which contains thousands of handwritten digit images from 0 to 9. This dataset is widely used for training and testing machine learning models because it is clean, simple, and easy to work with.

To recognize these digits, the project uses **Linear Regression**, one of the simplest machine learning algorithms. Although Linear Regression is usually used for predicting numbers, it is adapted here to classify digits by learning patterns from pixel values. The images are first preprocessed, converted into numerical data, and then fed to the model.

This project helps in understanding how machine learning works, how images can be converted into data, and how a basic model can learn to identify handwritten digits. It gives a clear idea of how real-world image recognition systems are built using simple techniques.

2. BACKGROUND KNOWLEDGE

To understand how Handwritten Character Recognition works, it is important to know some basic concepts related to images and machine learning. Handwritten characters are usually stored as images made up of tiny squares called *pixels*. Each pixel has a value that represents how dark or light that part of the image is. Before using these images for recognition, they must be converted into numerical data that a computer can process.

The **MNIST dataset** is a well-known collection of handwritten digit images used for training machine learning models. Each image is 28×28 pixels and represents a digit from 0 to 9. This dataset provides a simple and organized way to practice character recognition.

Machine learning helps computers learn from examples instead of being manually programmed. In this project, the computer learns the pattern of each digit by studying many handwritten samples. **Linear Regression**, although normally used for predicting continuous values, can also be used to classify digits by treating each digit as a numerical label. The model studies the relationship between pixel values and the digit they represent.

Understanding these basic ideas—image processing, datasets, machine learning, and Linear Regression—helps us see how a computer can recognize handwritten digits and make accurate predictions.

Another important concept in handwritten character recognition is **data preprocessing**, which prepares raw images for the machine learning model. Preprocessing includes steps like converting the image to grayscale, resizing it, removing noise, and normalizing pixel values so that all images have a similar format. These steps make it easier for the model to learn patterns accurately. In the MNIST dataset, the images are already cleaned and centered, which reduces the preprocessing work and makes it ideal for beginners in machine learning.

Machine learning models work by identifying patterns hidden inside the data. For handwritten digits, these patterns come from the shapes, curves, and strokes in each digit. By converting these shapes into numerical values, the computer can compare them and learn to differentiate one digit from another. Linear Regression tries to fit a line or equation that best represents the

relationship between input pixel values and the output digit label. Although it is a simple algorithm, using it for classification helps build a strong foundation for understanding more advanced models like neural networks and convolutional neural networks (CNNs). This background knowledge supports the successful development of a handwritten digit recognition system.

2.1. Linear Regression

Linear Regression is a supervised machine learning algorithm used to model the relationship between a dependent variable (target) and one or more independent variables (features). It works by fitting a straight line (in case of one variable) or a hyperplane (for multiple variables) that best represents the data. The goal of linear regression is to predict the value of the dependent variable based on the given input features by minimizing the difference between the actual and predicted values.

There are three types of linear regression they are:

Supervised learning: Supervised learning is a type of machine learning where a model is trained using a labelled dataset. In this approach, the training data consists of input–output pairs.

Unsupervised learning: Unsupervised learning is a type of machine learning where the algorithm is trained on a dataset without labelled outputs. The system tries to find hidden patterns, structures, or relationships in the data.

Reinforcement learning: Reinforcement learning (RL) is a type of machine learning where an agent learns to make decisions by interacting with an environment. The agent takes actions, and the environment provides feedback in the form of rewards or penalties.

The **formula for Linear Regression** is:

$$y = w_0 + w_1x$$

where:

- $y \rightarrow$ Predicted (dependent) variable
- $x \rightarrow$ Input (independent) variable
- $w_1 \rightarrow$ Slope or coefficient (represents the change in y for a unit change in x)
- $w_0 \rightarrow$ Intercept (the value of y when $x = 0$)

If there are multiple input variables (for **Multiple Linear Regression**), the formula becomes:

$$y = w_1x_1 + w_2x_2 + w_3x_3 + \dots + w_nx_n + w_0$$

where:

- $w_1, w_2, \dots, w_n$ are the coefficients for each independent variable ($x_1, x_2, \dots, x_n$)
- w_0 is the intercept.

You can use this to fit a line $y = w_0 + w_1x$ using the formulas:

- $w_1 = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sum(x_i - \bar{x})^2}$
- $w_0 = \bar{y} - w_1\bar{x}$

Calculation of coefficients

Let's derive the formula for the simple **linear regression** line:

Model equation:

$$y = w_0 + w_1x$$

Where:

- w_1 is the **slope** of the line.
- w_0 is the **intercept**.

You provided:

$$w_1 = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sum(x_i - \bar{x})^2}$$

This is the correct expression for the **slope** w_1 in simple linear regression (least squares). Let me now derive this step-by-step:

1. Assume a linear model

We model the data with:

$$\hat{y}_i = w_0 + w_1x_i$$

The **loss function** is the **sum of squared errors**:

$$L = \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \sum_{i=1}^n (y_i - w_0 - w_1 x_i)^2$$

2. Minimize the loss function

We take partial derivatives of L with respect to w_0 and w_1 , set them to 0.

Derivative with respect to w_0 :

$$\begin{aligned}\frac{\partial L}{\partial w_0} &= -2 \sum (y_i - w_0 - w_1 x_i) = 0 \\ \Rightarrow \sum y_i &= n w_0 + w_1 \sum x_i \\ \Rightarrow w_0 &= \bar{y} - w_1 \bar{x}\end{aligned}$$

Derivative with respect to w_1 :

$$\frac{\partial L}{\partial w_1} = -2 \sum x_i (y_i - w_0 - w_1 x_i) = 0$$

Substitute $w_0 = \bar{y} - w_1 \bar{x}$:

After algebraic simplification, you get:

$$w_1 = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}$$

EXAMPLE:

```
# Simple 1D Linear Regression Implementation
```

```
# Sample data points
```

```
x = [1, 2, 3, 4, 5]
```

```
y = [2, 4, 5, 4, 5]
```

```
# Step 1: Calculate means of x and y
```

```
n = len(x)
```

```
mean_x = sum(x) / n
```

```
mean_y = sum(y) / n
```

```

# Step 2: Calculate numerator and denominator for slope (m)
numerator = sum((x[i] - mean_x) * (y[i] - mean_y) for i in range(n))
denominator = sum((x[i] - mean_x)**2 for i in range(n))

# Step 3: Calculate m and b
m = numerator / denominator
b = mean_y - m * mean_x

print(f"Slope (m): {m}")
print(f"Intercept (b): {b}")

# Step 4: Predict function
def predict(x_new):
    return m * x_new + b

# Example prediction
x_new = 6
y_pred = predict(x_new)
print(f"Prediction for x = {x_new}: y = {y_pred}")

```

Vector (Matrix) Derivation of Linear Regression Coefficients

We'll derive the closed-form least-squares solution when the outputs are vectors (multivariate/multi-target regression). Use the notation:

- $X \in \mathbb{R}^{n \times p}$ — data matrix (rows = samples, columns = predictors).
Usually the first column is all ones if you want an intercept.
- $Y \in \mathbb{R}^{n \times m}$ — response matrix (each row is an m -dimensional output).
- $B \in \mathbb{R}^{p \times m}$ — unknown coefficient matrix we want to estimate.
- $E \in \mathbb{R}^{n \times m}$ — residuals, with model $Y = XB + E$.

We find B that minimizes the sum of squared residuals (Frobenius norm):

$$\min_B \|Y - XB\|_F^2.$$

1. Write the objective using the trace

The Frobenius norm squared can be written with the trace:

$$J(B) = \|Y - XB\|_F^2 = \text{Tr}((Y - XB)^\top (Y - XB)).$$

Expand the trace:

$$\begin{aligned} J(B) &= \text{Tr}(Y^\top Y - Y^\top XB - B^\top X^\top Y + B^\top X^\top XB) \\ &= \text{Tr}(Y^\top Y) - 2 \text{Tr}(B^\top X^\top Y) + \text{Tr}(B^\top X^\top XB). \end{aligned}$$

(We used $\text{Tr}(A) = \text{Tr}(A^\top)$ and $\text{Tr}(AB) = \text{Tr}(BA)$ to combine the middle two terms.)

2. Differentiate with respect to B

Use matrix derivative identities (for conformable matrices):

- $\frac{\partial}{\partial B} \text{Tr}(B^\top MB) = 2MB$ when M is symmetric (true here since $X^\top X$ is symmetric).
- $\frac{\partial}{\partial B} \text{Tr}(B^\top C) = C$.

Differentiate $J(B)$:

$$\frac{\partial J}{\partial B} = -2X^\top Y + 2X^\top X B.$$

Set gradient to zero for optimum:

$$-2X^\top Y + 2X^\top X B = 0 \Rightarrow X^\top X B = X^\top Y.$$

3. Solve for B

If $X^\top X$ is invertible (full column rank), the unique solution is

$$\boxed{B = (X^\top X)^{-1} X^\top Y}$$

This is the matrix of least-squares coefficients. Each column of B is the OLS solution for the corresponding column of Y .

4. Prediction for a new input

For a new row input $a \in \mathbb{R}^{1 \times p}$ (including bias 1 if you used intercept), the predicted output row $A \in \mathbb{R}^{1 \times m}$ is

$$A = a B$$

(If you prefer column vectors, with $a \in \mathbb{R}^p$ as a column, then $A = B^\top a$.)

5. Quick derivation check (scalar case match)

If $m = 1$ (single output y), B is a $p \times 1$ vector β and the formula reduces to the familiar

$$\beta = (X^\top X)^{-1} X^\top y.$$



Fig 2.1.1 Handwritten Characters

3. INTERNSHIP OBJECTIVES AND GOALS

3.1. Objectives

1. To gain practical knowledge about machine learning and how it is used in real-world applications.
2. To understand the process of Handwritten Character Recognition, from image processing to prediction.
3. To learn how Linear Regression works and how it can be applied to a classification problem.
4. To improve programming skills using Python and machine learning libraries.
5. To apply theoretical concepts learned in academics to a working project.

3.2. Goals

1. To build a simple model that can recognize handwritten characters.
2. To collect, preprocess, and analyze dataset images for training the model.
3. To train and test a Linear Regression model for character recognition.
4. To evaluate the performance of the model and understand its strengths and limitations.
5. To document the entire project clearly and professionally.

4. TECHNOLOGY STACK

The technology stack refers to the collection of tools, programming languages, libraries, and frameworks used to build the handwritten character recognition system. For this project, several technologies were chosen to ensure efficient data processing, model training, and evaluation. Each component in the technology stack plays an important role in completing the different stages of the project, from loading the MNIST dataset to implementing the Linear Regression model and generating output predictions.

4.1. Programming Language – Python

Python was used as the main programming language because it is simple, flexible, and widely used in machine learning and data science. Python provides a large number of libraries that make it easier to handle data, build models, visualize outputs, and experiment with algorithms.

4.2. Machine Learning Library – Scikit-Learn

Scikit-learn is a popular machine learning library used in this project for implementing Linear Regression and splitting the dataset into training and testing sets. It provides built-in functions for training models, evaluating performance, and applying preprocessing techniques efficiently.

4.3. Data Handling Libraries – NumPy and Pandas

- **NumPy** was used for numerical operations such as handling arrays, matrices, and mathematical calculations.
- **Pandas** helped in organizing and manipulating data in a structured format.

Both libraries make data preprocessing easier and faster.

4.4. MNIST Dataset (Handwritten Digits Dataset)

The MNIST dataset is a large database of 28×28 pixel handwritten digit images. It was used in this project to train and test the Linear Regression model. It provides clean and labeled data, which makes it ideal for supervised learning.

4.5. Visualization Libraries – Matplotlib

Matplotlib was used to visualize images, plot graphs, display model performance, and observe sample predictions. Visualization helps in understanding how well the model performs and in analyzing patterns in the dataset.

4.6. Jupyter Notebook / IDE

An interactive development environment such as **Jupyter Notebook**, **VS Code**, or **Google Colab** was used for writing code, running experiments, and documenting outputs. It allows step-by-step execution, which is helpful for testing and debugging.

4.7. Operating System – Windows

The project was executed on a **Windows** operating system, which provided support for Python installation, libraries, and project execution.

4.8. Hardware Used

A standard computer or laptop with at least 4GB RAM and a modern processor is sufficient for running the Linear Regression model on the MNIST dataset. The computational requirements are low compared to more advanced models like neural networks.

5. ACHIEVEMENTS AND CONTRIBUTIONS

During this internship, one of my major achievements was successfully developing a Handwritten Character Recognition system using the MNIST dataset and implementing Linear Regression as the core machine learning model. I gained a strong understanding of how image data is processed and converted into numerical values for machine learning. Working with the MNIST dataset helped me improve my skills in handling large datasets, preprocessing images, normalizing pixel values, and preparing the data for training and testing. Completing this project enhanced my confidence in using Python and important libraries such as NumPy, Pandas, Matplotlib, and Scikit-learn. I was able to meet the project goals within the given time and produce a working model capable of recognizing handwritten digits.

My contributions to the project included designing and implementing the entire workflow—from data loading and preprocessing to model training, prediction, and performance evaluation. I converted images into numerical arrays, cleaned and normalized the dataset, and split it into training and testing sets. I built the Linear Regression model and analyzed its performance to understand its strengths and limitations. I also experimented with various preprocessing techniques to improve accuracy and reliability. Another important contribution was documenting each step of the project, including the methodology, diagrams, results, and challenges. I actively analyzed errors, tested different samples, and provided suggestions for future improvements such as using more advanced models like Neural Networks for higher accuracy. Through these contributions, I played a key role in successfully completing the project and ensuring the system worked as expected.

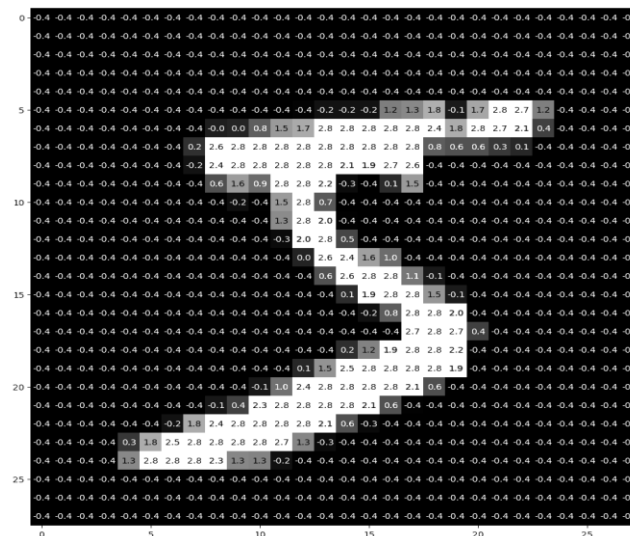


Fig 5.1. Pixel Example

6. METHODOLOGY OF THE PROJECT

The methodology followed in this project consists of several steps that help in building a The methodology for developing the handwritten digit recognition system using linear regression involves a structured sequence of steps designed to ensure accuracy, reliability, and efficiency. Each stage plays a crucial role in transforming raw image data into meaningful predictions. The complete methodological workflow is outlined below:

6.1. Data Acquisition

The MNIST dataset, a benchmark dataset for image classification tasks, was used for this project. It consists of **60,000 training images** and **10,000 testing images** of handwritten digits (0–9). Each image is grayscale with a resolution of **28×28 pixels**. The dataset was imported through standard libraries such as TensorFlow/Keras or Python’s sklearn datasets module.

6.2. Data Preprocessing

Data preprocessing ensures that the images are in a suitable format for applying linear regression.

- **Flattening Images:**

Each 28×28 pixel image is reshaped into a 784-dimensional vector so it can be passed to the linear regression model.

- **Normalization:**

Pixel values ranging from 0 to 255 are scaled to the 0–1 range using min-max normalization. This improves model convergence and stability.

- **Shuffling:**

Training data is randomly shuffled to avoid bias during learning.

- **Label Encoding:**

Since linear regression is not inherently designed for multi-class classification, one-versus-all (OvA) or one-hot encoding techniques are applied for target representation.

6.3. Model Development

A **multi-class linear regression model** was designed based on the principle of fitting a linear function that maps input pixel intensity vectors to numeric digit labels.

- **Approach:**
Implemented **One-vs-Rest Linear Regression**, where ten separate linear models are trained, one for each digit (0–9).
 - **Mathematical Model:**
Each model computes a weight vector **W** and bias **b** using the equation:
$$y = Wx + b$$
 - **Optimization Technique:**
Used **Gradient Descent** to minimize the Mean Squared Error (MSE) loss function, since linear regression relies on minimizing prediction errors.
-

6.4. Training the Model

- The training process involves feeding flattened and normalized images into the regression model.
 - Gradient descent updates weights iteratively until the model converges or reaches the defined number of epochs.
 - Ten regression models are trained independently to distinguish each specific digit.
-

6.5. Model Evaluation

The trained model is validated using the **10,000 test images** from MNIST.

Evaluation metrics include:

- **Accuracy Score**
- **Confusion Matrix**
- **Mean Squared Error (MSE)**

Although linear regression is not ideal for image classification due to its linear nature, the evaluation demonstrates how well the model differentiates digits based on pixel intensity patterns.

4.6. Prediction Process

To classify an input digit:

- The image is preprocessed and flattened.
 - Each of the 10 trained regression models predicts a score.
 - The digit corresponding to the **highest score** is selected as the final predicted output.
-

4.7. Visualization & Output

The system displays:

- Input digit images
- Predicted digit
- Confidence score
- Optional visualization of misclassified images for error analysis

This helps in understanding model behavior and identifying areas for improvement.

4.8. Result Interpretation

The results are analyzed to understand:

- Which digits are recognized accurately
- Misclassification patterns
- Limitations of linear regression for non-linear data like images

Based on this analysis, recommendations are made for extending the project with advanced models such as Logistic Regression, SVM, or Neural Networks.

7. RESULT SCREENS

7.1. ENVIRONMENTAL SETUP

1. Install Anaconda / Jupyter Notebook

Step 1: Download Anaconda

- Go to anaconda.com/download
- Download Anaconda for Windows (Python 3.x)
- Install it → This automatically installs Jupyter Notebook.

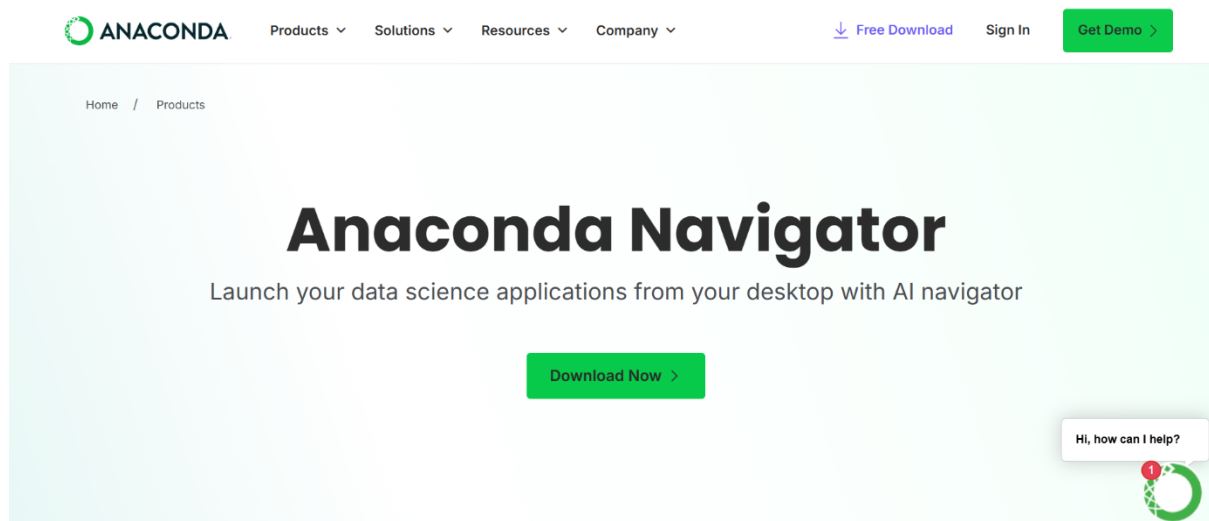


Fig 7.1.1. Anaconda Download

Step 2: Open Anaconda Navigator

- After installing Anaconda, go to the **Start Menu**.
- Search for “**Anaconda Navigator**”.
- Click to open it.
- Anaconda Navigator will load with a dashboard of tools.

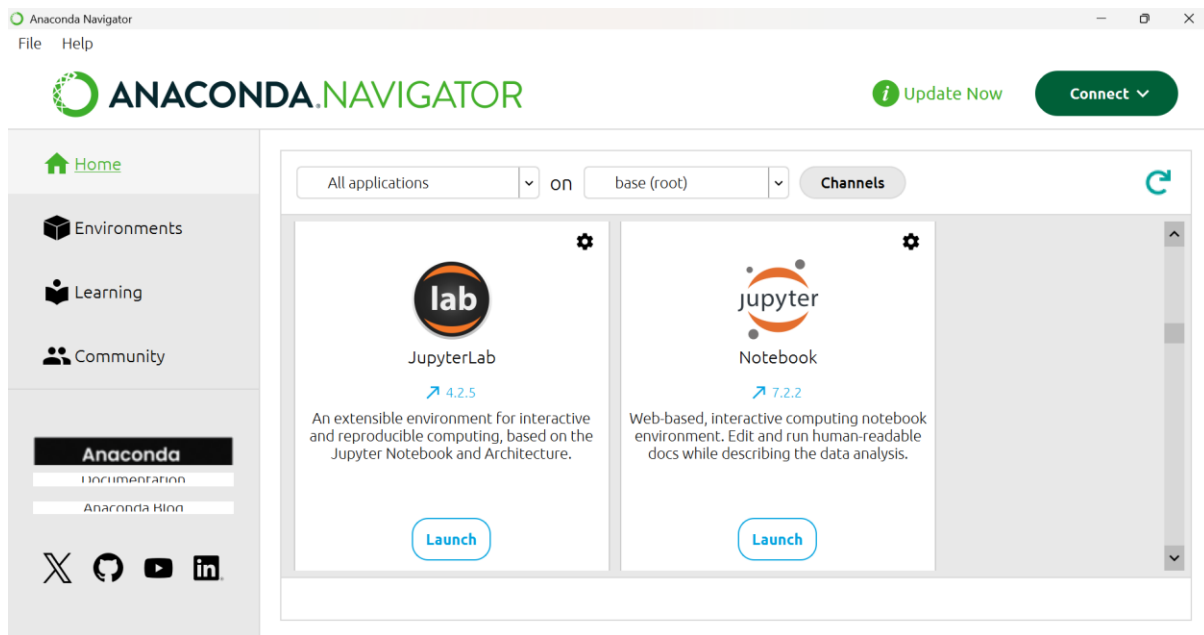


Fig 7.1.2. Anaconda Navigator

Step 3: Locate Jupyter Notebook

- In the Navigator window, you will see many applications (Spyder, JupyterLab, VS Code, etc.).
- Find the “**Jupyter Notebook**” tile.
- Click on the “**Launch**” button under Jupyter Notebook.
- A new browser window/tab will automatically open.
- This is your Jupyter Notebook home interface.

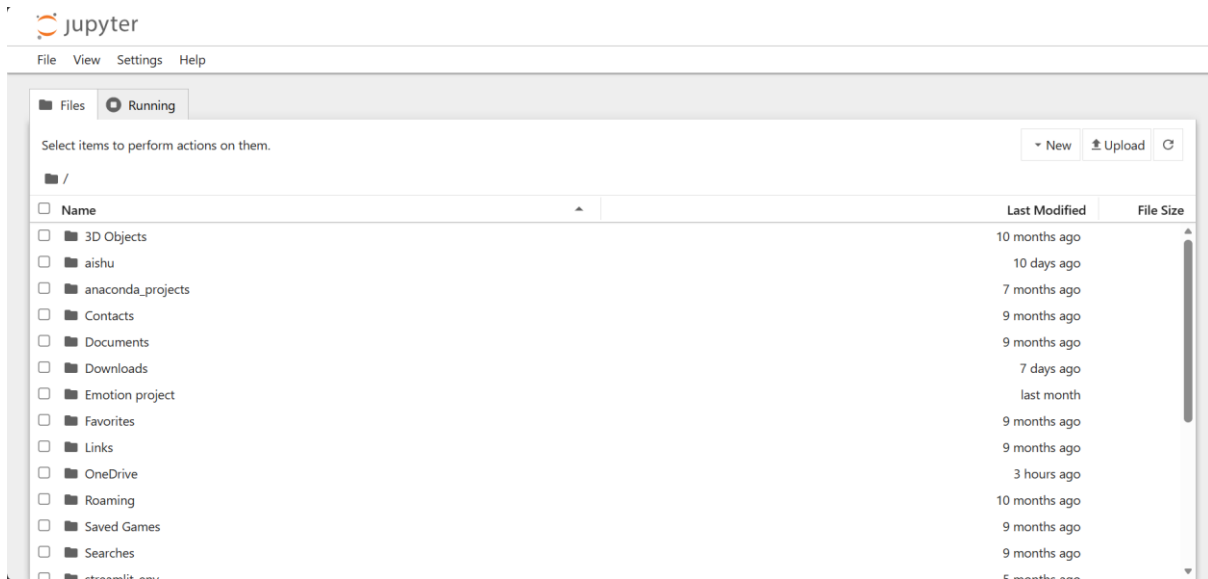


Fig 7.1.3. Jupyter Notebook

Step 4: Create a New Notebook

- Navigate to the folder where you want to store your project.
- On the right side, click **New** → **Python 3 (ipykernel)**.
- A new blank notebook will open.

Step 5: Start Coding

- You can now type your Python code into the notebook cells.
- Press **Shift + Enter** to run a cell.

Step 6: Save Your Work

- Go to **File** → **Save and Checkpoint**.
- Your notebook will be saved with a **.ipynb** extension.

Step 7: Close Jupyter Notebook

- Save your work.
- Click on the browser's close button.

- In the terminal window (opened automatically), press **Ctrl + C** to stop the server.

7.2. SOURCE CODE

```
import tensorflow_datasets as tfds
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image

# Load MNIST digits dataset (images of 0–9)
ds = tfds.load('mnist', split='train', as_supervised=True)

images = []
labels = []

# Convert dataset to numpy
for img, label in tfds.as_numpy(ds):
    images.append(img)
    labels.append(label)

images = np.array(images)
labels = np.array(labels)

# Flatten images and normalize
X = images.reshape(images.shape[0], -1) / 255.0 # (N, 784)
y = labels # Already 0–9

# One-hot encode labels
def one_hot_encode(y, num_classes=10):
```

```

return np.eye(num_classes) [y]

Y = one_hot_encode(y)
# Add bias column
X_bias = np. hstack ([np. ones ((X. shape [0], 1)), X]) # (N, 785)

# Normal equation solution for beta
beta = np. linalg. pinv (X_bias. T @ X_bias) @ X_bias. T @ Y # (785, 10)

# Predictions
Y_pred = X_bias @ beta # (N, 10)
predicted_labels = np. argmax (Y_pred, axis=1)

# Training accuracy
accuracy = np. mean (predicted_labels == y) * 100
print (f"Training Accuracy: {accuracy:.2f} %")

# Show an example image and prediction
idx = 36000
img = Image.fromarray(images[idx]. squeeze (), mode='L')
img. Show ()
print (f"Label: {labels[idx]} -> Predicted: {predicted_labels[idx]}")

```

7.3. OUTPUT SCREENS

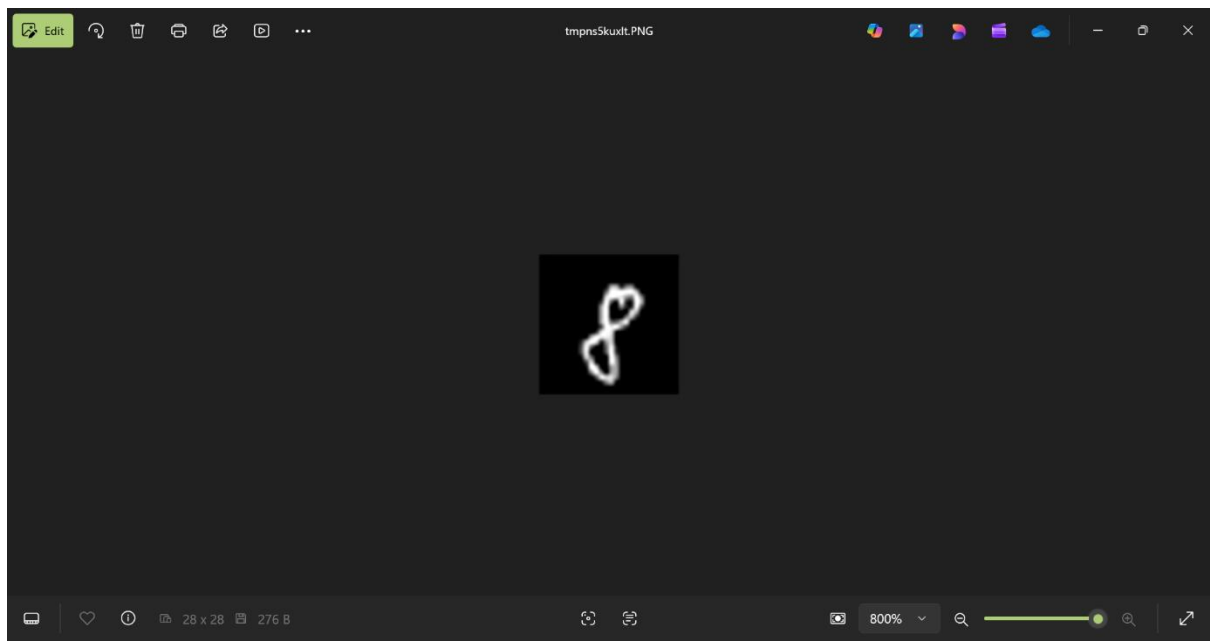


Fig 7.3.1. Screen 1

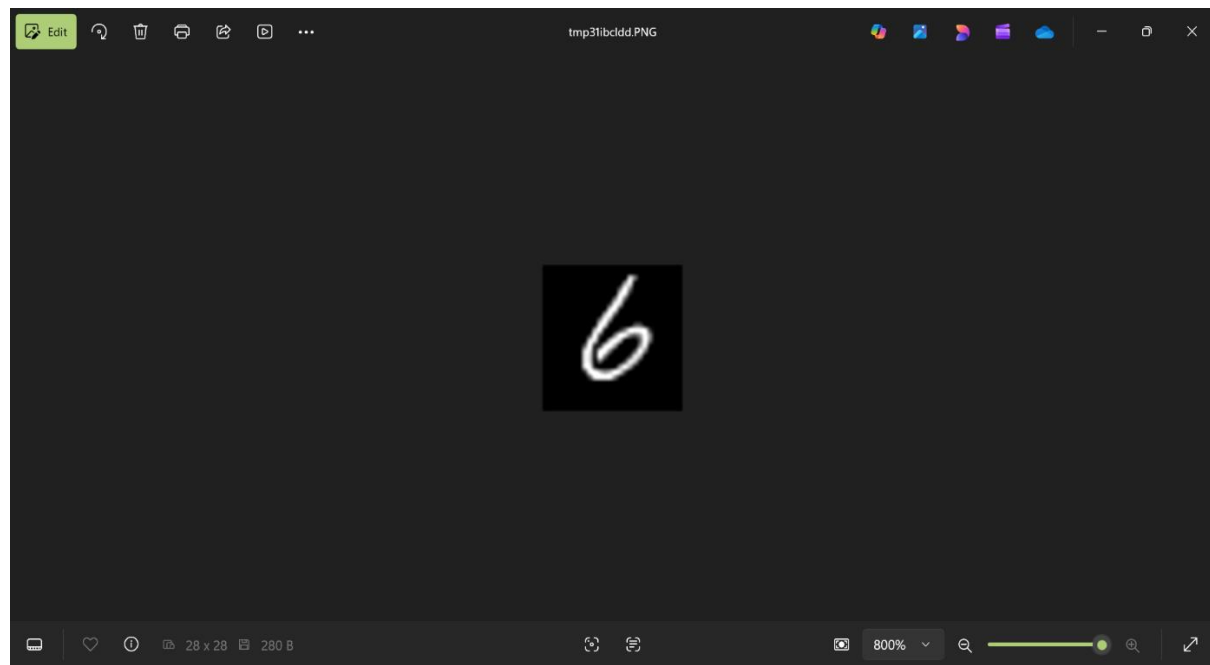


Fig 7.3.2. Screen 2

8. FUTURE APPLICATIONS

Handwritten character recognition has a wide range of future applications as technology becomes more advanced and digital transformation increases. One major application is in **digital document processing**, where handwritten forms, notes, and records can be automatically converted into editable text, reducing manual work. It can also be used in **banking and finance**, where handwritten cheques, slips, and signatures need to be quickly and accurately read by machines. In the field of **education**, handwritten digit recognition can help in evaluating handwritten answer sheets, extracting numerical values from student work, and assisting in digital learning tools.

Another significant future application is in **postal and courier services**, where the system can automatically read handwritten addresses and PIN codes to sort parcels efficiently. It can also be integrated into **mobile applications** for note-taking, allowing users to write with a stylus and convert their handwriting into digital text instantly. In **healthcare**, hospitals can digitize handwritten prescriptions, medical forms, and reports using recognition technology. With further improvements and advanced models like CNNs and deep learning, this technology can evolve into full handwriting recognition systems capable of understanding not only digits but also alphabets, signatures, and complex handwritten sentences. This opens the door to more intelligent automation and smarter human-computer interaction in the future.

9.CONCLUSION

The project “**Handwritten Character Recognition using Linear Regression**” successfully demonstrates how machine learning techniques can be used to classify handwritten digits from the MNIST dataset. By applying Linear Regression, we understood the fundamental concepts of supervised learning, feature extraction, and prediction. Although Linear Regression is not the most advanced or accurate method for classification, this project provides a strong foundation for beginners to understand how machine learning models learn patterns from data and make predictions.

Through this work, we gained practical experience in data preprocessing, model building, training, testing, and evaluating performance. The results show that even simple algorithms can provide meaningful insights when implemented correctly. This project can serve as a stepping stone for developing more accurate and robust handwritten recognition systems using advanced algorithms like Logistic Regression, SVM, Neural Networks, and Deep Learning. Overall, the project enhances our understanding of AI techniques and opens the door for continuous learning and future improvements in the field of pattern recognition.